

予約リマインダー・フォローメッセージ 送信動作 監査レポート

【完全版】

作成日: 2026-09-11 / 統合元: Claude Code レポート (リマインダー・フォローメッセージ_送信動作監査レポート.md) + Codex レポート (..._Codex.md)

このレポートは 2 つの監査結果を 1 つにまとめ、「誰が・何を・どの順で」やるかが分かるように整理したものです。

> **2026-09-11 更新 (対応状況)** > - 2 章の確認 SQL を本番・staging で実行済み。本番のフォロー cron が staging の URL を呼んでいた (毎時 401)、staging のリマインダー cron がテンプレの URL・仮キーのままだった。どちらも Dashboard から登録し直して解消。本番のリマインダー cron は正常 (毎時 200) > - 「設定した時刻に届かない」と報告のあった 2 店舗は LINE チャネルアクセストークン未設定が原因 (予約フォーム未導入の店舗のため問題なし) > - 修正パッケージ A~D はすべて実装済み (マイグレーション 20260911000000_message_delivery_reliability.sql、PR は feature/message-delivery-reliability)。仕様は Q1-B / Q2-A / Q3-A で確定 > - 残作業 (🤖): staging → 本番へのマイグレーション適用と 2 つの Edge Function の再デプロイ (リリース手順参照)

0.3 行まとめ

- 日付・時刻の計算そのものは正しい (両レポート一致)。「何日前」「送信時刻」「基準日 + N 日後」は設定どおりに計算されています。
- それでも「送られない・遅れる・二重に届く」は起こり得ます。原因は (a) Supabase 側の cron 登録・鍵の状態 (本番で未確認) と (b) コードの弱点 (取り逃しの回収なし・二重送信防止なし・通信例外で全体停止 など) の 2 系統です。
- (a) は **あなたが本番の SQL Editor で確認 SQL を実行すれば確定**します。(b) は **すべて私 (Claude) がコードで直せます**。仕様の判断が必要なものだけ 5 章で選んでください。

1. 役割分担の早見表

記号	意味
🤖 Claude	私に任せてください。コード修正・テスト・ドキュメントまで行い、PR を出します
👤 あなた	Supabase Dashboard での SQL 実行・鍵の入力・デプロイなど、私が触れない作業
🗨️ 要相談	仕様 (どう動くのが正しいか) をあなたに決めてもらうもの。選択肢を用意しました

#	問題	重要度	出典	担当
P1	本番 cron が毎時になっていない / 鍵がプレスホルダー・新形式で 401	高	両方	🤖 確認 → 🤖 登録し直し
P2	リマインダーは指定の 1 時間を逃すと二度と送られない（回収なし）	高	Codex A01	🤖
P3	リマインダーは同じ予約に何度でも送れる（送信記録・排他なし）	高	両方	🤖
P4	LINE 通信で例外が出ると残りの予約が全部送られない（両 Function）	高	Codex A05	🤖
P5	フォローの二重送信防止（樂觀ロック）が送信中の再起動に弱い	高	Codex A04	🤖
P6	フォロー設定（何日後・時刻・基準日）を変えても既存の予約に反映されない	高	Codex A03	🤖（既定方針あり）
P7	再予約をキャンセルしても元のフォローが復活しない	高	Codex A06	🤖（既定方針あり）
P8	フォロー予定の作成失敗 / 同時作成で予定が欠落・重複する可能性	高	Codex A07	🤖
P9	cron → Function の HTTP タイムアウト未指定（既定 5 秒）	中	両方	🤖 SQL 作成 → 🤖 実行
P10	フォローは cron が 5 分起動なので「12:00」設定でも 12:05 以降に届く	中	Codex B01	🤖
P11	予約のメニュー等を編集しただけで送信待ちフォローが消える（理由も誤り）	中	Codex B02	🤖
P12	リマインダー設定の保存時にサーバー検証がない（12:30 等が入ると一致しない）	中	Codex B05	🤖
P13	取得件数の上限（1000 行）を超えると取り逃す可能性	中	Codex B06	🤖
P14	LINE チャンネルアクセストークン未設定の店舗が黙ってスキップされる	低	Claude 4-6	🤖（画面に警告）
P15	フォローを ON にする前の予約には送られない	中・仕様	両方	📁
P16	希望日時（第一～第三希望）は第一希望日基準、未確定（pending）にも送る	中・仕様	両方	📁
P17	「予約受付日から」基準だと来店前に「ご来店ありがとうございました」が届き得る	中・仕様	Codex B04	📁

#	問題	重要度	出典	担当
P18	送信ログが無く「送ったか」を後から確認できない（リマインダー）	中	両方	 （P2・P3 と同時に解決）
P19	LINE の 200 応答 = 到達ではない（ブロック等）。リトライキ-未使用	低	Codex B07	 （P3・P5 と同時）

2. まず最初にやること（🧑 あなた・15 分）

コード修正の前に、**本番が今どうなっているか**を確定させます。これが分からないと「直したのに届かない」が続きます。

Supabase Dashboard → **wakuwakusystemsharing-prod** → SQL Editor で、次を上から順に実行し、結果（スクリーンショットで OK）を私に送ってください。**すべて読み取りだけで、データは変わりません。**

2-1. cron の登録状態（最重要）

```
select jobid, jobname, schedule, active,  
       substring(command from 'https://[^/ ]+supabase[.]co') as target_origin,  
       position('<SERVICE_ROLE_KEY>' in command) > 0 as placeholder_key,  
       position('timeout_milliseconds' in command) > 0 as explicit_timeout  
from cron.job  
where command like '%send-reminders%' or command like '%send-follow-messages%'  
order by jobid;
```

見るところ:

- `send-reminders` を呼ぶジョブが **1 本だけ**で `schedule = 0 * * * *`（毎時）か。 `0 10 * * *` なら毎日 1 回のまま
- `placeholder_key` が **false** か。true なら鍵が仮のまま（全件 401）
- `target_origin` が `https://tpuqjpdasxfwsvjcbum.supabase.co`（本番）か。staging の URL なら誤登録
- 同じ URL を呼ぶジョブが 2 本あれば二重送信の原因

2-2. cron が Function を呼んだ結果

```
select id, status_code, timed_out, error_msg is not null as has_transport_error,  
       created at time zone 'Asia/Tokyo' as created_jst  
from net._http_response  
order by created desc limit 50;
```

見るところ: `status_code` が 200 なら呼べている。401 / 403 なら鍵の問題。 `timed_out = true` が並ぶなら P9。行が無い場合は 6 時間以上 cron が動いていないか、 `pg_net` が無効（ただし保持期間切れの可能性もあるので 2-3 と合わせて判断）。

2-3. cron の実行履歴

```
select j.jobname, d.status, d.start_time at time zone 'Asia/Tokyo' as start_jst  
from cron.job_run_details d join cron.job j on j.jobid = d.jobid  
where j.command like '%send-reminders%' or j.command like '%send-follow-messages%'  
order by d.start_time desc limit 60;
```

見るところ: 1 時間ごとに `succeeded` が並ぶか。1 日 1 回なら P1。

2-4. 店舗の設定と列の存在

```
select id, name, reminder_enabled, reminder_days_before, reminder_time,  
       coalesce(length(trim(line_channel_access_token)) > 0, false) as has_token,  
       follow_enabled, follow_base, follow_days_after, follow_time  
from stores order by name;
```

見るところ: エラーになれば列が無い（マイグレーション未適用）。届かない店舗の `reminder_enabled` / `has_token` / `reminder_time` の書式（HH:00 か）。

2-5. フォローの状態

```
select status, skip_reason, count(*) from follow_messages group by 1, 2 order by 1;  
select count(*) as overdue from follow_messages where status = 'scheduled' and scheduled  
_at <= now();
```

見るところ: `overdue` が増え続けていれば `cron` が動いていない。

> staging でも同じ SQL を実行しておく、本番との差が分かります。

2-6. 結果が「毎日 1 回」or「鍵が仮」だった場合の直し方 (🤖)

次を **本番の `service_role` キー (eyJ で始まる JWT 形式。Project Settings → API → `service_role`)** に置き換えて実行します。同じ名前なので上書きされます。

```
select cron.schedule(
  'send_reservation_reminders',
  '0 * * * *',
  $$
    select net.http_post(
      url := 'https://tpuqjpdasxfwsvjcbum.supabase.co/functions/v1/send-reminders',
      headers := jsonb_build_object(
        'Content-Type', 'application/json',
        'Authorization', 'Bearer ここに本番の service_role キーを貼る'
      ),
      timeout_milliseconds := 60000
    );
  $$
);
```

フォロー側 (`send_follow_messages`) も `placeholder_key = true` なら同様に、URL を `send-follow-messages`、スケジュールを `5 * * * *` にして登録し直してください。

注意: P3 が直るまで、Function を手動で何度も叩かないでください (同じ予約に複数回届きます)。

3. 私 (Claude) に任せられる修正の一覧

「これをやって」と言ってもらえれば着手します。まとめて 1 つの PR にもできますし、分けることもできます。

修正パッケージ A: リマインダーの信頼性 (P2・P3・P4・P12・P13・P18・P19) — 工数 1 日

内容	どう直すか	直ると何がかわるか
送信記録テーブル <code>reminder_logs</code> を追加	<code>reservation_id</code> + 対象日 を一意キーに、 <code>sent</code> / <code>failed</code> / <code>skipped</code> と理由・LINE 応答を保存	「送ったか」を店舗ごとに後から確認できる。 同じ予約に二度送らない

内容	どう直すか	直ると何が変わるか
取り逃しの回収	「現在の時 = 送信時刻」の完全一致をやめ、「対象日で、送信時刻を 過ぎていて 、まだ送信記録が無い予約」を送る	cron が 1 回止まっても、その日のうちなら次の回で送られる。送信時刻の変更・遅れて入った予約も拾う
回収の期限	対象日を過ぎた（= 予約日当日になった）ものは送らない	来店後に「明日の予約」が届く事故を防ぐ
排他取得 + LINE リトライキー	送る前に記録行を <code>sending</code> で確保し、 <code>X-Line-Retry-Key</code> にその ID を使う	手動実行・重複 cron でも二重に届かない
1 件ごとの例外処理 + 通信タイムアウト	<code>fetch</code> を予約ごとの try/catch で囲み、10 秒でタイムアウト	1 件の通信失敗で残りが止まらない
ページング	店舗・予約の取得を 1000 件ずつ	大量店舗でも取り逃さない
保存時の検証 + DB 制約	<code>reminder_time</code> は HH:00、 <code>reminder_days_before</code> は 1~7、テンプレートの型を <code>PUT</code> で検証。DB にも CHECK	<code>12:30</code> などの不正値が入らない
空トークンの除外	抽出条件を <code><> ''</code> にし、画面に「トークン未設定」の警告	設定 ONなのに届かない店舗が分かる

> 既存の `send-reminders` を書き換えますが、送る条件は「設定した日の設定した時刻以降」で変わりません。文面も変わりません。

修正パッケージ B: フォローの信頼性（P5・P8・P10・P11・P13） — 工数 半日～1 日

内容	どう直すか
送信中の排他	<code>status = 'sending' + claimed_at</code> （10 分で失効）で行を確保。結果更新は確保した実行だけが行う。 <code>X-Line-Retry-Key</code> = 行 ID
通信例外の扱い	例外も試行回数に数え、取得条件に <code>attempt_count < 3</code> を付ける。1 件の例外で残りを止めない
予定の欠落防止	予約作成時の「差し替え + 作成」を DB 関数（1 トランザクション）にする。送信 Function 側に「直近 3 日の対象予約で予定行が無いものを作る」補完を追加
5 分ずれの解消	cron は毎時 5 分のまま（リマインダーと起動をずらすため）。画面の説明を「送信は毎時 5 分にまとめて行います。設定時刻から 10 分以内に届きます」に変更し、表示と実際を一致させる
内容編集で消える問題	予約日が変わらない編集では予定を触らない。再計算で過去になった予定は「次回実行で送る」扱いにし、 <code>store_disabled</code> と分ける
1 回 200 件の上限	期限到来分を全件処理（時間内に終わらなければ次回に持ち越し、順序保証）

修正パッケージ C: 設定変更・予約変更との整合性（P6・P7） — 工数 半日

既定の方針で実装します。違う動きがよければ 5 章で教えてください。

内容	既定の方針
フォロー設定を変えたとき（P6）	その店舗の 未送信 予定をすべて新設定で計算し直す。計算結果が過去なら次回実行で送る。送信済み・見送り済みは触らない
再予約をキャンセルしたとき（P7）	同じ顧客の「差し替えられた」予定のうち、元の予約が有効で予定日が未来のものを 1 件だけ <code>scheduled</code> に戻す

修正パッケージ D: 運用まわり（P1・P9 の補助） — 工数 1 時間

内容
cron 登録 SQL のテンプレートに <code>timeout_milliseconds := 60000</code> を追加し、リリース手順に「Dashboard から手動登録が必須」「鍵は JWT 形式」を明記
店舗編集画面に「トークン未設定」「送信は HH:00 以降」の注意書き
顧客詳細の予約履歴に「リマインダー: 送信済み / 失敗」バッジ（パッケージ A の記録を表示）

私ができないこと

- 本番・staging の DB を直接読む・書く（SQL の実行は 🤖）
- Service Role キーの入力・cron の登録（🤖）
- Edge Function のデプロイ（`npx supabase functions deploy ...`）。前回と同じくコマンドをお渡ししますので実行してください。私の環境から実行できる場合もありますが、その際は事前に確認します
- LINE 実機での受信確認（👤）。テスト用店舗と受信用アカウントが必要）

4. 推奨する進め方

順	誰	やること	所要
1	🤖	2 章の確認 SQL を本番（できれば staging も）で実行し、結果を共有	15 分
2	🤖	結果に応じて 2-6 で cron を登録し直す（毎時 + 本物の鍵 + タイムアウト）	10 分
3	📁	5 章の 3 つの仕様を決める（既定のままでも可）	5 分
4	💻	修正パッケージ A～D を実装（テスト付き）→ PR	2～3 日
5	🤖	staging にマージ → マイグレーション実行 → 2 つの Function をデプロイ	20 分

順	誰	やること	所要
6	👤 + 🖨️	テスト店舗で「同じ時間に 2 回実行しても 1 通」「cron を 1 回止めても次で届く」「設定変更後の予定が更新される」を確認	30 分
7	👤	本番へ（PR → マイグレーション → デプロイ → cron 更新）	30 分

手順 1〜2 だけでも、「設定した時刻に届かない」の大半は解消する見込みです。3〜7 は「止まっても回収される」「二重に届かない」を保証するための作業です。

5. 決めてもらいたい仕様（📁 選択肢）

Q1. フォローを ON にしたとき、すでに入っている未来の予約はどうしますか？（P15）

- A. 今のまま: ON 後に入った予約だけ対象（実装済み。説明文を画面に追加）
- B. ON に切り替えた時点で、未来の予約（LINE 経由）にも予定を作る（推奨）
- C. 未来の予約に加えて、過去 N 日以内の来店にも作る（予定日が未来になるものだけ）

Q2. 未確定（pending）の予約・希望日時モードの予約はどう扱いますか？（P16）

- A. 今のまま: キャンセル以外すべてに送る（第一希望日基準）（現状）
- B. 店舗ごとに「確定（confirmed）した予約だけ送る」スイッチを追加（推奨）
- C. 希望日時モードの予約だけ確定必須にする

Q3. 「予約受付日から N 日後」で来店前に届くケースはどうしますか？（P17）

- A. 今のまま（店舗が文面で調整する。画面に注意書きを追加）（推奨・最小）
- B. 受付日基準のときは、予定日が来店日より前なら来店日の翌日にずらす
- C. 受付日基準のときのデフォルト文面を「ご予約ありがとうございました」に変える

返信は「Q1-B、Q2-B、Q3-A」のように記号だけで大丈夫です。返信が無ければ **推奨** で進めます。

6. 補足: 両レポートの見解が違った点と、この完全版での扱い

論点	Claude	Codex	完全版の扱い
原因の最有力は cron / 鍵か	最有力と判定	実データが無いので 順位付け不可	順位は付けず「最初に確認すべき候補」とし、2 章で確定させる
db push で毎時になるか	ならない	SQL が通れば毎時にはなるが鍵が 仮のまま	Codex が正確。「毎時だが仮キー」と「毎日のまま」の両方を 2-1 で判別

論点	Claude	Codex	完全版の扱い
フォローの二重送信防止	楽観ロックで防止	送信中の再起動には効かない（再現）	Codex が正確。パッケージ B で <code>sending</code> 状態に変更
フォローの失敗は 3 回で終了	3 回	HTTP エラーは 3 回、通信例外は無限	Codex が正確。パッケージ B で修正
HTTP タイムアウト 5 秒の影響	10 件超で打ち切りの可能性	環境依存で断定不可	断定せず 、 <code>timeout_milliseconds</code> を付けるのは害が無いので推奨
<code>net._http_response</code> が空 = cron 停止	そう判断	保持 6 時間なので断定不可	Codex が正確。2-3 の実行履歴と合わせて判断

Codex が再現に使ったスクリプトは `docs/codex-message-audit/reproduce.cjs`、結果は `results.json` にあります。

7. 参照

- `docs/リマインダー・フォローメッセージ_送信動作監査レポート.md`（Claude 版・詳細な確認 SQL 6 本）
- `docs/リマインダー・フォローメッセージ_送信動作監査レポート_Codex.md`（Codex 版・再現 14 シナリオ）
- `docs/フォローメッセージ機能_実装設計.md` / `docs/フォローメッセージ_リリース手順.md`
- `supabase/functions/send-reminders/index.ts` / `supabase/functions/send-follow-messages/index.ts`
- `supabase/migrations/20250102000000_prod_cron_job.sql` / `20260411100001_update_cron_hourly.sql` / `20260908000000_add_follow_message.sql`